Dynamic Programming → LeetCode

Learning & Implementation Guide

Topics covered: Longest Increasing Subsequence, Longest Common Subsequence, Longest Common Substring, Longest Palindromic Subsequence, Minimum Cost Path, Unique Paths, Minimum Edit Distance, and Subset Sum DP.

The guide focuses on simple, standard DP implementations for learning rather than runtime-optimized solutions.

1. Longest Increasing Subsequence — LeetCode 300

Problem statement

Given an integer array, find the length of the longest subsequence in which every next number is greater than the previous number. A subsequence does not need to be contiguous; elements can be skipped while keeping their order.

Example

Example: nums = [10, 9, 2, 5, 3, 7, 101, 18]. One longest increasing subsequence is [2, 5, 7, 101], so the answer is 4.

DP understanding

dp[i] = length of the longest increasing subsequence ending at nums[i]. For every earlier index j, if nums[j] < nums[i], nums[i] can extend the subsequence ending at j.

Java implementation

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];

        for (int i = 0; i < n; i++) {
            dp[i] = 1;
        }

        int max = 1;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            max = Math.max(max, dp[i]);
        }

        return max;
    }
}
```

Time: O(n²) Space: O(n)

2. Longest Common Subsequence — LeetCode 1143

Problem statement

Given two strings, return the length of their longest subsequence common to both strings. Characters do not have to be next to each other, but their order must remain the same.

Example

Example: text1 = "abcde", text2 = "ace". The LCS is "ace", so the answer is 3.

DP understanding

dp[i][j] = LCS length of the first i characters of text1 and the first j characters of text2. If the current characters match, use the diagonal value + 1. Otherwise use the larger of the top and left values.

Java implementation

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[m][n];
    }
}
```

Time: O(mn) Space: O(mn)

3. Longest Common Substring — LeetCode 718

Problem statement

LeetCode 718 asks for the maximum length of a subarray common to two integer arrays. This uses the same DP pattern as longest common substring because matching elements must be contiguous.

Example

Example: nums1 = [1,2,3,2,1], nums2 = [3,2,1,4,7]. The longest common contiguous part is [3,2,1], so the answer is 3.

DP understanding

dp[i][j] = length of the common contiguous sequence ending at nums1[i-1] and nums2[j-1]. If they match, use diagonal + 1. If they do not match, set the cell to 0 because continuity is broken.

Java implementation

```
class Solution {
    public int findLength(int[] nums1, int[] nums2) {
        int m = nums1.length;
        int n = nums2.length;

        int[][] dp = new int[m + 1][n + 1];
        int max = 0;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (nums1[i - 1] == nums2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                    max = Math.max(max, dp[i][j]);
                } else {
                    dp[i][j] = 0;
                }
            }
        }

        return max;
    }
}
```

Time: O(mn) Space: O(mn)

4. Longest Palindromic Subsequence — LeetCode 516

Problem statement

Given a string, return the length of its longest subsequence that is a palindrome. Characters can be skipped, but their original order is preserved.

Example

Example: s = "bbbab". A longest palindromic subsequence is "bbbb", so the answer is 4.

DP understanding

dp[i][j] = longest palindromic subsequence length inside s[i..j]. Every single character has length 1. If s[i] == s[j], use the inside range + 2. Otherwise discard one end and take the larger result.

Java implementation

```
class Solution {
    public int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int i = n - 2; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i + 1][j - 1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[0][n - 1];
    }
}
```

Time: O(n²) Space: O(n²)

5. Minimum Cost Path — LeetCode 64

Problem statement

Given a grid of non-negative numbers, start at the top-left cell and reach the bottom-right cell. You can move only right or down. Return the minimum possible path sum.

Example

Example: grid = [[1,3,1],[1,5,1],[4,2,1]]. The minimum path sum is 7.

DP understanding

dp[i][j] = minimum cost to reach cell (i,j). For an inner cell, the only previous cells are above and left, so take the smaller cost and add the current grid value.

Java implementation

```
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;

        int[][] dp = new int[m][n];

        dp[0][0] = grid[0][0];

        for (int i = 1; i < m; i++) {
            dp[i][0] = dp[i - 1][0] + grid[i][0];
        }

        for (int j = 1; j < n; j++) {
            dp[0][j] = dp[0][j - 1] + grid[0][j];
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = grid[i][j] +
                           Math.min(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

Time: O(mn) Space: O(mn)

6. Unique Paths — LeetCode 62

Problem statement

A robot starts at the top-left of an m × n grid and must reach the bottom-right. It can move only right or down. Return the number of possible paths.

Example

Example: m = 3, n = 7. The answer is 28.

DP understanding

dp[i][j] = number of ways to reach cell (i,j). Every cell except the first row and first column can be reached from either above or left, so add those two counts.

Java implementation

```
class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];

        for (int j = 0; j < n; j++) {
            dp[0][j] = 1;
        }

        for (int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

Time: O(mn) Space: O(mn)

7. Minimum Edit Distance — LeetCode 72

Problem statement

Convert word1 into word2 using the minimum number of insertions, deletions, and replacements. Return the minimum number of operations.

Example

Example: word1 = "horse", word2 = "ros". The answer is 3.

DP understanding

dp[i][j] = minimum operations to convert the first i characters of word1 into the first j characters of word2. If characters match, no operation is needed. Otherwise choose the cheapest of insert, delete, and replace, then add 1.

Java implementation

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            dp[i][0] = i;
        }

        for (int j = 0; j <= n; j++) {
            dp[0][j] = j;
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = Math.min(
                        dp[i - 1][j - 1],
                        Math.min(dp[i - 1][j], dp[i][j - 1])
                    ) + 1;
                }
            }
        }

        return dp[m][n];
    }
}
```

Time: O(mn) Space: O(mn)

8. Subset Sum DP — LeetCode 416

Problem statement

LeetCode 416 asks whether an array can be partitioned into two subsets with equal sum. If the total sum is even, the problem becomes: can we find a subset whose sum is totalSum / 2?

Example

Example: nums = [1,5,11,5]. Total = 22, so target = 11. The subset [1,5,5] has sum 11 and the remaining [11] also has sum 11. Answer: true.

DP understanding

dp[i][j] = whether a subset using the first i numbers can make sum j. For each number, either do not take it, or take it. If it is too large for j, we can only skip it. Sum 0 is always possible by choosing nothing.

Java implementation

```
class Solution {
    public boolean canPartition(int[] nums) {
        int n = nums.length;
        int totalSum = 0;

        for (int i = 0; i < n; i++) {
            totalSum += nums[i];
        }

        if (totalSum % 2 != 0) {
            return false;
        }

        int target = totalSum / 2;

        boolean[][] dp = new boolean[n + 1][target + 1];

        for (int i = 0; i <= n; i++) {
            dp[i][0] = true;
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= target; j++) {
                if (nums[i - 1] <= j) {
                    dp[i][j] =
                        dp[i - 1][j] ||
                        dp[i - 1][j - nums[i - 1]];
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }

        return dp[n][target];
    }
}
```

Time: O(n × target) Space: O(n × target)

Quick LeetCode DP Map

Topic	LeetCode	Core DP idea
Longest Increasing Subsequence	300	dp[i] = LIS ending at i
Longest Common Subsequence	1143	match → diagonal + 1; mismatch → max(top,left)
Longest Common Substring	718	match → diagonal + 1; mismatch → 0
Longest Palindromic Subsequence	516	match ends → inside + 2; mismatch → max
Minimum Cost Path	64	current + min(top,left)
Unique Paths	62	top + left
Minimum Edit Distance	72	min(insert, delete, replace) + 1
Subset Sum DP	416	skip OR take current number

Learning note: First understand what dp[i] or dp[i][j] represents, then identify the previous states it depends on. The direction of the loops is determined by those dependencies.